

BAB V

FUNCTION & METHOD

5.1. Tujuan

1. Praktikan dapat memahami konsep dasar *function* dan *method*.
2. Praktikan dapat memahami perbedaan *function* dan *method*.
3. Praktikan dapat memahami fungsi dari *function* dan *method*.
4. Praktikan dapat mengimplementasikan *function* dan *method* dalam bahasa pemrograman C++, Python, Java, dan PHP.
5. Praktikan mampu merancang dan menyusun solusi dari suatu permasalahan sederhana menggunakan pendekatan *function* atau *method*.

5.2. Dasar Teori

5.3.1. Function

Function merupakan sebuah subrutin independen yang merangkum sekumpulan instruksi komputasi secara spesifik untuk mengeksekusi tugas tertentu. Dalam arsitektur perangkat lunak, fungsi bertindak sebagai *mini-program* yang secara struktural terisolasi dari blok eksekusi program utama. Hal ini memungkinkan pemanggilan algoritma secara berulang-ulang (*code reusability*), sehingga *programmer* tidak perlu melakukan penulisan baris kode yang sama secara redundan.

Implementasi fungsi sangat krusial untuk menerapkan prinsip modularitas, yaitu teknik memecah keseluruhan program yang kompleks menjadi blok-blok sistem yang lebih kecil dan terstruktur. Pendekatan ini memberikan dua keuntungan utama:

1. **Optimalisasi Keterbacaan dan Efisiensi:** Struktur program menjadi jauh lebih rapi, terorganisir, dan efisien secara alur logika.
2. **Kemudahan *Debugging*:** Ketika terjadi *bug* atau *syntax error*, *programmer* dapat dengan cepat melakukan pelacakan dan mengidentifikasi letak *error-code* pada blok fungsi tertentu, tanpa harus mengevaluasi keseluruhan program utama.

Secara umum dalam berbagai bahasa pemrograman, fungsi diklasifikasikan menjadi dua tipe utama berdasarkan mekanisme nilai kembaliannya:

1. *Return Type (Non-Void Type)*
Fungsi jenis ini dirancang untuk memproses suatu operasi dan wajib mengembalikan sebuah nilai (*value*) baru ke blok kode yang memanggilnya. Karakteristik utamanya adalah keharusan menggunakan *keyword* *return* di dalam *block code* eksekusinya dan nilai akhir yang dikembalikan oleh instruksi *return* tersebut harus divalidasi agar selaras (memenuhi *type-checking*) dengan tipe data yang telah dideklarasikan pada *header* fungsi tersebut.
2. *Non-Return Type (Void Type)*

Fungsi tipe ini secara eksplisit dideklarasikan menggunakan tipe data. Sesuai namanya, subrutin ini murni hanya mengeksekusi serangkaian *statement* atau aksi (seperti manipulasi *state* atau mencetak *output*) tanpa mengembalikan nilai kembalian apa pun setelah instruksinya selesai dieksekusi. Oleh karena operasi ini tidak menghasilkan *output* data baru ke *caller*, maka penggunaan *keyword return* tidak diperlukan di dalam blok operasinya.

5.3.2. Method

Pada dasarnya, *method* itu sama saja dengan fungsi (*function*). Bedanya, istilah "*method*" ini lebih sering dipakai kalau kita sedang menggunakan konsep Pemrograman Berorientasi Objek (PBO). PBO sendiri adalah sebuah metode pemrograman yang fokus utamanya menggunakan *object* (objek) dan *class* (kelas). Jadi, secara sederhana, *method* adalah sebuah fungsi yang posisinya berada di dalam sebuah *class*.

Karena *method* sebenarnya adalah fungsi juga, maka cara menulis atau mendefinisikannya di dalam kode program sama persis seperti saat kita membuat fungsi biasa. Aturan kodenya tinggal menyesuaikan dengan bahasa pemrograman apa yang sedang kita pakai.

5.3.3. Method vs Function

Cara paling gampang untuk melihat bedanya antara *function* (fungsi) dan *method* adalah dari bagaimana kita memanggilnya saat ingin digunakan di dalam program.

- Cara Memanggil Function:

Kalau kita mau menjalankan sebuah fungsi, kita bisa langsung menyebutkan nama fungsi tersebut. Kalau fungsinya butuh data tambahan (biasa disebut argumen), kita tinggal memasukannya ke dalam kurung. **Contoh:** Kalau ada fungsi bernama *cetak*, kita cukup menuliskan kodenya langsung seperti ini:

```
cetak() ;
```

- Cara Memanggil Method:

Di sisi lain, *method* tidak bisa dipanggil langsung dengan namanya. Kita harus memanggilnya lewat sebuah objek (*instance object*) yang merupakan perwujudan atau perwakilan dari *class* tempat *method* itu berada. Biasanya, kita menggunakan tanda titik (.) untuk menyambungkan nama objek dengan nama *method*-nya. **Contoh:** Kalau `cetak` ini adalah sebuah *method*, kita butuh sebuah objek dulu. Misalnya kita membuat objek dengan nama `printer`. Nah, kode yang harus kita tulis untuk memanggilnya bukan lagi `cetak()` ;, melainkan menjadi `printer.cetak()` ;.

5.3.4. Penulisan dan Penggunaan Function

Secara umum, sebuah *function* atau *method* memiliki beberapa bagian penting, yaitu *access modifier*, tipe data, nama fungsi, parameter, dan *return statement*. Setiap bagian memiliki fungsi yang berbeda dalam proses pembuatan dan penggunaan fungsi.

1. *Access Modifier* (Opsional)

Access modifier digunakan untuk mengatur tingkat akses terhadap *class*, *variable*, maupun *method*. Dalam pemrograman berorientasi objek, *access modifier* menentukan bagian mana dari program yang dapat mengakses suatu *variable* atau *method*. Pada Java, terdapat beberapa jenis *access modifier*, yaitu:

a. *Public*

Modifier public menunjukkan bahwa *class*, *variable*, atau *method* dapat diakses dari *class* lain, termasuk *class* yang berada di luar *package*.

b. *Private*

Modifier private menunjukkan bahwa *variable* atau *method* hanya dapat diakses dari dalam *class* tempat *variable* atau *method* tersebut dideklarasikan.

c. *Protected*

Modifier protected memungkinkan *variable* atau *method* diakses dari *class* itu sendiri, *class* lain dalam *package* yang sama, serta *subclass* atau *class* turunannya.

Selain ketiga *modifier* tersebut, Java juga memiliki akses *default* atau *package-private*. Akses ini digunakan ketika suatu *variable* atau *method* dideklarasikan tanpa menuliskan *access modifier*. Dalam kondisi tersebut, *variable* atau *method* dapat diakses oleh *class* lain yang berada dalam *package* yang sama.

2. Tipe Data atau *Return Type*

Tipe data digunakan untuk menentukan jenis nilai yang dihasilkan oleh sebuah fungsi. Jika fungsi menghasilkan suatu nilai, maka tipe data nilai tersebut harus ditentukan, seperti *int*, *double*, *String*, dan sebagainya. Sebaliknya, jika fungsi tidak mengembalikan nilai, maka digunakan tipe *void*.

3. Nama Fungsi

Nama fungsi digunakan untuk mengidentifikasi dan memanggil fungsi tersebut. Penamaan fungsi sebaiknya menggambarkan proses atau tugas yang dilakukan oleh fungsi agar kode lebih mudah dipahami.

4. Parameter dan Argumen (opsional)

Parameter merupakan *variable* yang dideklarasikan pada bagian fungsi dan digunakan untuk menerima nilai ketika fungsi dipanggil. Parameter hanya dapat digunakan di dalam fungsi tempat parameter tersebut dideklarasikan.

Parameter dituliskan di dalam tanda kurung setelah nama fungsi. Sebuah fungsi dapat memiliki satu atau lebih parameter yang dipisahkan menggunakan tanda koma.

Argumen merupakan nilai yang diberikan ketika sebuah fungsi dipanggil. Nilai argumen tersebut akan diterima dan digunakan oleh parameter untuk menjalankan proses yang terdapat di dalam fungsi.

5. *Keyword Return* (kondisional)

Keyword return digunakan pada fungsi yang memiliki *return type* selain *void*. Fungsi *return* digunakan untuk mengembalikan hasil dari proses yang dilakukan oleh fungsi kepada bagian program yang memanggilnya. Secara umum, penulisan fungsi pada beberapa bahasa pemrograman dapat dituliskan sebagai berikut.

a. Java dan C++

```
<tipe_data> <nama_fungsi>(parameter1, parameter2) {
```

```
Proses1;  
Proses2;  
return;  
}
```

b. Python

```
def <nama_fungsi>(parameter1, parameter2):  
    proses1  
    proses2  
    return
```

c. PHP

```
function <nama_fungsi>(parameter1, parameter2) {  
    proses1;  
    proses2;  
    return;  
}
```

Untuk menggunakan atau menjalankan sebuah fungsi, fungsi dapat dipanggil dengan menuliskan nama fungsi beserta argumen yang diperlukan. Contohnya sebagai berikut.

```
nama_fungsi(argumen1, argumen2);
```

Argumen yang diberikan saat pemanggilan fungsi akan diteruskan ke parameter untuk kemudian digunakan dalam proses yang terdapat di dalam fungsi.

5.3. Percobaan

5.3.5. Function pada Java

Pada percobaan 1, buat *project* baru dengan nama Modul4_kelxx, ganti xx dengan nomor kelompok. Lalu buat *class* baru atau ubah nama *class Main* yang muncul menjadi matematika. Lalu masukkan *source code* berikut.

```
public class matematika {  
  
    static void penjumlahan(int a, int b) {  
        int c = a + b;  
        System.out.println("Hasil penjumlahan: " + c);  
    }  
  
    static void pengurangan() {  
        int c = 10 - 20;  
        System.out.println("Hasil pengurangan: " + c);  
    }  
  
    static int pembagian(int a, int b) {  
        int c = a / b;  
        return c;  
    }  
  
    static int perkalian() {  
        int c = 10 * 20;  
        return c;  
    }  
  
    public static void main(String[] args) {  
        penjumlahan(10,20);  
  
        pengurangan();  
  
        System.out.println("Hasil perkalian: " + perkalian());  
  
        int d = pembagian(100,20);  
        System.out.println("Hasil pembagian: " + d);  
    }  
}
```

```
Hasil penjumlahan: 30  
Hasil pengurangan: -10  
Hasil perkalian: 200  
Hasil pembagian: 5  
  
Process finished with exit code 0
```

Gambar 5. 1 Output Function pada Java

Keterangan:

- a. Fungsi `penjumlahan()`
 - Fungsi tersebut tidak mengembalikan *value*
 - Fungsi ini merupakan ***non-return type***, ditandai dengan tipe data ***void***
 - Fungsi tersebut memiliki parameter dengan tipe data *integer*, dengan nama variabelnya a dan b
- b. Fungsi `pengurangan()`
 - Fungsi tersebut tidak mengembalikan *value*
 - Fungsi ini merupakan ***non-return type***, ditandai dengan tipe data ***void***
 - Fungsi tersebut tidak memiliki parameter
- c. Fungsi `pembagian()`
 - Fungsi ini mengembalikan *value* dengan tipe data *integer*
 - Fungsi ini merupakan ***return type*** yang ditandai dengan tipe data *integer*
 - Fungsi ini memiliki parameter dengan tipe data *integer*
- d. Fungsi `perkalian()`
 - Fungsi ini mengembalikan *value* dengan tipe data *string*
 - Fungsi ini merupakan ***return type*** yang ditandai dengan tipe data *string*
 - Fungsi ini tidak memiliki parameter

5.3.6. Method pada Java

Pada percobaan 2, buat *class* baru dengan nama **intro**, lalu masukkan *source code* berikut.

```
public class intro {  
  
    public void sapa() {  
        System.out.println("Haloo Worldd!!");  
    }  
  
    public String perkenalan(String nama, String kota, String hobi) {
```

```

        return "Namaku " + nama + ", Aku dari " + kota + " dan
        hobiku adalah " + hobi;
    }

    public void umur(int umur) {
        System.out.println("Aku berumur "+ umur + " tahun");
    }
}

```

Lalu tambahkan kode berikut pada *class* matematika dari percobaan 1, didalam *function* main.

```

        System.out.println();
        System.out.println("-----");

        intro objek = new intro();

        objek.sapa();

        System.out.println(objek.perkenalan("Lanjar", "Blora", "Ndoding(kali)"));

        objek.umur(15);

```

Pada percobaan 2, memanggil *method* dengan membuat objek dari *class* Kemudian panggil fungsi-fungsi dari *class* di dalam *main function*.

```

-----
Haloo Worldd!!
Namaku Lanjar, Aku dari Blora dan hobiku adalah Ndoding(kali)
Aku berumur 15 tahun

Process finished with exit code 0

```

Gambar 5. 2 *Output Method* pada Java

5.3.7. Function pada C++

Pada percobaan 1, buat proyek dengan nama Modul4_kelxx, xx diganti dengan nomor kelompok masing-masing. Setelah itu masukkan *source code* berikut kedalam teks *editor*.

```

#include <iostream>
using namespace std;

void penjumlahan(int a, int b) {

```

```

    int c = a + b;
    cout << "Hasil penjumlahan: " << c <<endl;
}

void pengurangan() {
    int c = 10 - 20;
    cout << "Hasil pengurangan: " << c <<endl;
}

int pembagian(int a, int b) {
    int c = a / b;
    return c;
}

int perkalian() {
    int c = 10 * 20;
    return c;
}

int main() {

    penjumlahan(10,20);

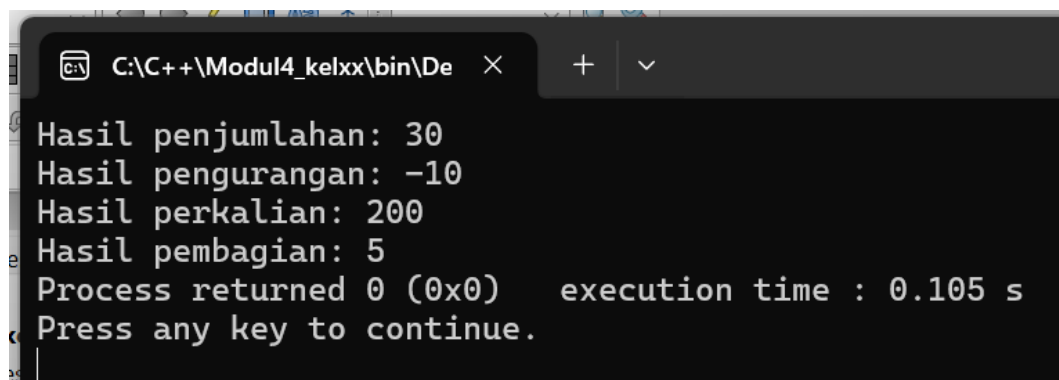
    pengurangan();

    perkalian();
    cout << "Hasil perkalian: " << perkalian() <<endl;

    int d = pembagian(100,20);
    cout << "Hasil pembagian: " << d;

    return 0;
}

```



The screenshot shows a terminal window with the following output:

```

C:\C++\Modul4_kelxx\bin\De
Hasil penjumlahan: 30
Hasil pengurangan: -10
Hasil perkalian: 200
Hasil pembagian: 5
Process returned 0 (0x0)    execution time : 0.105 s
Press any key to continue.

```

Gambar 5.3 *Output Function* pada C++

5.3.8. Method pada C++

Pada percobaan 2, buat proyek dengan nama Modul4_kelxx_method, xx diganti dengan nomor kelompok masing-masing. Setelah itu masukkan *source code* berikut kedalam teks *editor*.

```
#include <iostream>
using namespace std;

class matematika1 {
public:
    void penjumlahan(int a, int b) {
        int c = a + b;
        cout << "Hasil penjumlahan: " << c << endl;
    }

    void pengurangan() {
        int c = 10 - 20;
        cout << "Hasil pengurangan: " << c << endl;
    }
};

class matematika2 {
public:
    int pembagian(int a, int b) {
        int c = a / b;
        return c;
    }

    int perkalian() {
        int c = 10 * 20;
        return c;
    }
};

int main() {
    matematika1 objek1;

    objek1.penjumlahan(10,20);

    objek1.pengurangan();

    matematika2 objek2;

    cout << "Hasil perkalian: " << objek2.perkalian() << endl;

    int d = objek2.pembagian(100,20);
    cout << "Hasil pembagian: " << d;

    return 0;
}
```

Gambar 5. 4 *Output Method* pada C++

5.3.9. Function pada Python

Percobaan 1, buat file dengan nama percobaan1_kelompokXX.py, kalian bisa ganti XX dengan nomor kelompok kalian (contoh : 01, 10, dst). Setelah itu masukkan *source code* berikut ke *code editor*.

```
# 1. Non-return function (Void)
def sapa_pengunjung(nama, asal_daerah):
    print(f"Selamat datang {nama} dari {asal_daerah} di sistem
    pemantauan Desa Patakbanteng.")

# 2. Return function (Mengembalikan nilai yang sebenarnya)
def cek_status_suhu(suhu):
    print(f"Suhu yang tercatat: {suhu}°C")
    if suhu < 20:
        return "Status: Suhu terpantau dingin."
    else:
        return "Status: Suhu normal atau hangat."

# 3. Arbitrary function (Jumlah argumen bebas)
def catat_titik_cctv(*titik_lokasi):
    print("Lokasi CCTV yang sedang aktif:")
    for lokasi in titik_lokasi:
        print(f"- {lokasi}")

# 4. Anonymous function (Lambda)
# Misalnya untuk menghitung selisih data
hitung_selisih = lambda x, y: x - y

# Main program
sapa_pengunjung("Dafa", "Semarang")
print("-" * 30)
hasil_suhu = cek_status_suhu(18)
print(hasil_suhu)
print("-" * 30)
catat_titik_cctv("Gerbang Utama", "Balai Desa", "Area Parkir")
print("-" * 30)
print("Selisih pengunjung hari ini dan kemarin:",
    hitung_selisih(150, 120))
```

```

PS C:\Users\User\OneDrive\DOCUMENT HABIB\ASPRAK\PROGDAS\FUNCTION\python> & C:\Python314\p
rs/User/OneDrive/DOCUMENT HABIB/ASPRAK/PROGDAS/FUNCTION/python/percobaan1_kelompokXX.py"
Selamat datang Dafa dari Semarang di sistem pemantauan Desa Patakbanteng.
-----
Suhu yang tercatat: 18°C
Status: Suhu terpantau dingin.
-----
Lokasi CCTV yang sedang aktif:
- Gerbang Utama
- Balai Desa
- Area Parkir
-----
Selisih pengunjung hari ini dan kemarin: 30

```

Gambar 5.5 Percobaan 1 Python

Pada program variasi di atas, terdapat empat tipe fungsi yang digunakan, yaitu *non-return function*, *return function*, *arbitrary function*, dan *anonymous function*. *Non-return function* (seperti pada fungsi `sapa_pengunjung`) merupakan fungsi yang tidak mengembalikan nilai apa pun, melainkan hanya melakukan aksi tertentu seperti mencetak pesan sambutan pengunjung ke layar. Selanjutnya, *return function* (pada fungsi `cek_status_suhu`) mengembalikan nilai setelah melakukan suatu proses, di mana setelah fungsi mengevaluasi angka suhu, teks status suhu tersebut dikembalikan secara nyata kepada pemanggil fungsi. Selain itu, *arbitrary function* (pada fungsi `catat_titik_cctv`) adalah fungsi dengan jumlah parameter yang fleksibel. Jumlah parameter ini tidak ditentukan sebelumnya dan dideklarasikan dengan simbol asterisk (*) di awal nama parameter, sehingga memungkinkan kita untuk memasukkan sejumlah argumen titik lokasi CCTV yang bervariasi. Terakhir, *anonymous function* atau fungsi anonim adalah fungsi yang tidak diberi nama. Fungsi ini dideklarasikan menggunakan kata kunci `lambda`, yang memungkinkan pembuatan fungsi sederhana, seperti perhitungan selisih angka, tanpa harus menggunakan sintaks panjang seperti pada fungsi konvensional.

5.3.10. Method pada Python

Percobaan 2, buat file dengan nama `percobaan2_kelompokXX.py`, kalian bisa ganti XX dengan nomor kelompok kalian (contoh : 01, 10, dst). Setelah itu masukkan *source code* berikut ke *code editor*.

```

class MahasiswaTekkom:
    def __init__(self, nama, konsentrasi):
        self.nama = nama
        self.konsentrasi = konsentrasi

```

```

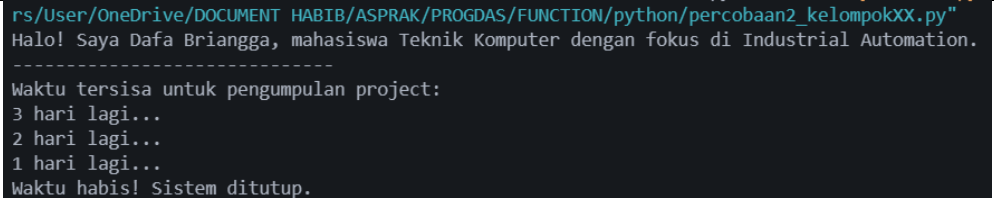
def profil(self):
    print(f"Halo! Saya {self.nama}, mahasiswa Teknik
Komputer dengan fokus di {self.konsentrasi}.")

def deadline_tugas(self, hari):
    print("Waktu tersisa untuk pengumpulan project:")
    while hari > 0:
        print(f"{hari} hari lagi...")
        hari -= 1
    print("Waktu habis! Sistem ditutup.")

obj = MahasiswaTekkom("Dafa Briangga", "Industrial Automation")

obj.profil()
print("-" * 30)
obj.deadline_tugas(3)

```



```

rs/User/OneDrive/DOCUMENT HABIB/ASPRAK/PROGDAS/FUNCTION/python/percobaan2_kelompokXX.py"
Halo! Saya Dafa Briangga, mahasiswa Teknik Komputer dengan fokus di Industrial Automation.
-----
Waktu tersisa untuk pengumpulan project:
3 hari lagi...
2 hari lagi...
1 hari lagi...
Waktu habis! Sistem ditutup.

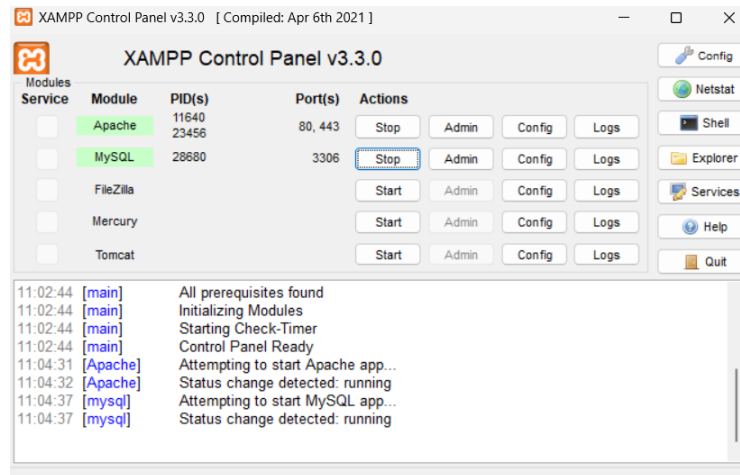
```

Gambar 5.6 Percobaan 2 Python

Dalam kode program variasi tersebut, terdapat sebuah kelas bernama `MahasiswaTekkom` yang memiliki dua metode utama, yaitu `profil` dan `deadline_tugas`. Metode `__init__` bertindak sebagai konstruktor yang bertugas menginisialisasi objek dengan atribut nama dan konsentrasi. Selanjutnya, metode `profil` berfungsi untuk mencetak pesan perkenalan yang secara otomatis menyertakan nilai dari atribut nama dan konsentrasi tersebut. Sementara itu, metode `deadline_tugas` digunakan untuk mengatur *countdown* atau hitung mundur dari jumlah hari yang diinputkan sebagai parameter, mencetak setiap sisa hari melalui proses *looping* hingga mencapai angka nol. Pada bagian pemanggilan program, kelas `MahasiswaTekkom` digunakan untuk melakukan instansiasi, yaitu menciptakan sebuah wujud objek baru yang menampung data nama dan konsentrasi tersebut. Setelah objek tercipta, barulah metode `profil` dan `deadline_tugas` dapat dipanggil melalui perantara objek itu dengan menggunakan tanda titik, yang mana secara langsung membedakan cara pemanggilan *method* dibandingkan dengan fungsi biasa.

5.3.11. Function pada PHP

Sebelum memulai percobaan 1, aktifkan terlebih dahulu Apache melalui XAMPP Control Panel.



Gambar 5.7 Xampp Control Panel

Lalu buat folder dengan nama progdas lalu buat *file* baru dengan nama percobaan1_kelompokXX.php (ganti bagian “XX” dengan nomor kelompok kalian masing – masing) di dalam folder ‘xampp/htdocs/progdas’. Masukkan kode berikut pada teks *editor* kalian.

```
<?php
function jumlahkan($a, $b) {
    return $a + $b;
}

$x = 50;
$y = 30;
$hasil_return = jumlahkan($x, $y);

echo "Hasil function return: $hasil_return <br><hr>";

function cetak_jumlah($a, $b) {
    $hasil = $a + $b;
    echo "Hasil function non-return: $hasil";
}

cetak_jumlah($x, $y);
?>
```

Jika sudah simpan *file* kalian lalu buka melalui browser dengan mengakses ke ‘http://localhost/progdas/percobaan1_kelompokXX.php’. Hasil akan menampilkan perhitungan dengan dua jenis *function* : satu mengembalikan nilai (*return type*) dan satu lagi yang langsung menampilkan hasil (*non-return type*).

Hasil function return: 80

Hasil function non-return: 80

Gambar 5.8 *Output function* pada PHP

5.3.12. Method pada PHP

Pada bahasa PHP, Pemrograman Berbasis Objek memiliki sintaksis yang sedikit berbeda jika dibandingkan bahasa pemrograman yang lain. Untuk memanggil *method* menggunakan simbol panah (->) untuk memanggil *method* dari sebuah objek, bukan menggunakan titik.

Dalam percobaan 2, buat file baru bernama percobaan2_kelompokXX.php di dalam folder yang sama dengan percobaan 1. Lalu tambahkan *source code* berikut :

```
<?php
class MyClass {
    // Method dengan return type
    public function hitungAkar($nilai) {
        return sqrt($nilai);
    }

    // Method tanpa return type (menggabungkan data)
    public function cetakProfil($nama, $umur) {
        echo "Nama saya $nama <br>";
        echo "Umur saya $umur <br>";
        echo "Saya adalah seorang mahasiswa";
    }
}

// Pembuatan instance objek
$objek = new MyClass();

// Pemanggilan method menggunakan ->
echo "Akar kuadrat dari 121 adalah: " . $objek-
>hitungAkar(121) . "<br><hr>";

$objek->cetakProfil("Budi", 18);
?>
```

Simpan dan jalankan *file* di browser dengan mengakses 'http://localhost/progdas/percobaan2_kelompokXX.php'. Hasilnya program akan menampilkan *output* akar kuadrat dari angka yang sudah diberikan lalu ada nama, umur, serta pekerjaan yang dihasilkan oleh *method* dalam *class*

Akar kuadrat dari 121 adalah: 11

Nama saya Budi

Umur saya 18

Saya adalah seorang mahasiswa

Gambar 5.9 *Output* Percobaan 2 pada PHP